

Files, Exception Handling, Logging and Memory Management

Theory Questions

1. What is the difference between interpreted and compiled languages?

Interpreted language runs the code with the help of an interpreter. Compiled language first converts the program into machine code by a compiler. Python is generally called an interpreted language and C is a compiled language.

2. What is exception handling in Python?

Exception handling is used to handle errors in a program. It helps the program to show a proper message instead of stopping suddenly. Python uses try and except for this.

3. What is the purpose of the finally block in exception handling?

The finally block is used for code which should run at the end. It runs whether an error occurs or not. It is often used for closing files.

4. What is logging in Python?

Logging means saving information about what is happening in a program. It is useful for finding errors and checking the working of a program.

5. What is the significance of the `__del__` method in Python?

The `__del__` method is related to deleting an object. It can be called when an object is going to be removed. It can be used for cleanup, but it should not be depended on for important cleanup work.

6. What is the difference between import and from ... import in Python?

import imports a module. We use the module name to use its functions. from ... import imports a particular function or object from a module.

7. How can you handle multiple exceptions in Python?

We can use more than one except block. We can also handle more than one exception in one except block using a tuple.

8. What is the purpose of the with statement when handling files in Python?

The with statement is used to work with files safely. It automatically closes the file after the work is completed.

9. What is the difference between multithreading and multiprocessing?

Multithreading uses more than one thread in a program. Multiprocessing uses more than one process. Threads are useful for I/O tasks and processes can be useful for CPU-heavy tasks.

10. What are the advantages of using logging in a program?

Logging helps to find errors and understand the program. It keeps a record of important events. It is also useful when we need to check a program later.

11. What is memory management in Python?

Memory management means managing the memory used by a Python program. Python automatically manages most of the memory for us.

12. What are the basic steps involved in exception handling in Python?

First we write risky code inside try. Then except is used to handle the error. else can be used when there is no error and finally can be used for cleanup.

13. Why is memory management important in Python?

It helps the program use memory properly. Good memory management can reduce unnecessary memory use and can improve program performance.

14. What is the role of try and except in exception handling?

try contains the code which may give an error. except catches and handles that error. This stops the program from crashing because of that error.

15. How does Python's garbage collection system work?

Python removes objects which are no longer needed. It uses reference counting and garbage collection to free unused memory.

16. What is the purpose of the else block in exception handling?

The else block runs when the try block does not give any error. It is used for code which should run after successful execution.

17. What are the common logging levels in Python?

Common logging levels are DEBUG, INFO, WARNING, ERROR and CRITICAL. They show different levels of information and problems.

18. What is the difference between os.fork() and multiprocessing in Python?

os.fork() creates a child process and is mainly available on Unix systems. multiprocessing is a Python module used to create and manage processes and is more portable.

19. What is the importance of closing a file in Python?

Closing a file frees the file resource and makes sure the data is properly saved. Using with can close the file automatically.

20. What is the difference between file.read() and file.readline() in Python?

read() reads the file content, or the requested number of characters. readline() reads one line from the file.

21. What is the logging module in Python used for?

The logging module is used to create log messages. These messages can be saved in a file or shown on the screen.

22. What is the os module in Python used for in file handling?

The os module is used for working with files, folders and the operating system. For example, it can check if a file exists.

23. What are the challenges associated with memory management in Python?

Large objects can use a lot of memory. Too many objects can also make the program use more memory. Circular references can also create memory management problems.

24. How do you raise an exception manually in Python?

We use the raise keyword. Example: raise ValueError('Invalid value').

25. Why is it important to use multithreading in certain applications?

Multithreading can help when a program has to wait for tasks like file reading or internet requests. One thread can wait while another thread does some work.

Practical Questions

1. How can you open a file for writing in Python and write a string to it?

Solution:

```
with open("example.txt", "w") as file:
    file.write("Hello Python")

print("Data written")
```

Expected result:

Data written

2. Write a Python program to read the contents of a file and print each line.

Solution:

```
with open("example.txt", "r") as file:
    for line in file:
        print(line.strip())
```

Expected result:

The lines of the file are printed one by one.

3. How would you handle a case where the file doesn't exist while trying to open it for reading?

Solution:

```
try:
    with open("example.txt", "r") as file:
        print(file.read())
except FileNotFoundError:
    print("File not found")
```

Expected result:

File not found

4. Write a Python script that reads from one file and writes its content to another file.

Solution:

```
with open("source.txt", "r") as file:
    data = file.read()

with open("newfile.txt", "w") as file:
    file.write(data)

print("File copied")
```

Expected result:

File copied

5. How would you catch and handle division by zero error in Python?

Solution:

```
try:
    a = 10
    b = 0
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Expected result:

Cannot divide by zero

6. Write a Python program that logs an error message to a log file when a division by zero exception occurs.

Solution:

```
import logging

logging.basicConfig(filename="error.log",
                    level=logging.ERROR)
```

```
try:
    print(10 / 0)
except ZeroDivisionError:
    logging.error("Division by zero error")
    print("Error saved in log file")
```

Expected result:

Error saved in log file

7. How do you log information at different levels (INFO, ERROR, WARNING) in Python using the logging module?

Solution:

```
import logging

logging.basicConfig(level=logging.DEBUG)

logging.info("Program started")
logging.warning("This is warning")
logging.error("This is error")
```

Expected result:

INFO, WARNING and ERROR messages are displayed.

8. Write a program to handle a file opening error using exception handling.

Solution:

```
try:
    file = open("data.txt", "r")
    print(file.read())
    file.close()
except OSError:
    print("Error while opening file")
```

Expected result:

Error while opening file, if the file cannot be opened.

9. How can you read a file line by line and store its content in a list in Python?

Solution:

```
with open("example.txt", "r") as file:
    lines = file.readlines()

print(lines)
```

Expected result:

The file lines are stored in a list.

10. How can you append data to an existing file in Python?

Solution:

```
with open("example.txt", "a") as file:
    file.write("\nNew data")

print("Data added")
```

Expected result:

Data added

11. Write a Python program that uses a try-except block to handle an error when attempting to access a dictionary key that doesn't exist.

Solution:

```
student = {"name": "Anu", "marks": 80}

try:
    print(student["age"])
except KeyError:
    print("Key does not exist")
```

Expected result:

Key does not exist

12. Write a program that demonstrates using multiple except blocks to handle different types of exceptions.

Solution:

```
try:
    num = int(input("Enter number: "))
    print(10 / num)
except ValueError:
    print("Enter a valid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Expected result:

The suitable except block handles the error.

13. How would you check if a file exists before attempting to read it in Python?

Solution:

```
import os

if os.path.exists("example.txt"):
    with open("example.txt", "r") as file:
        print(file.read())
else:
    print("File does not exist")
```

Expected result:

File is read only when it exists.

14. Write a program that uses the logging module to log both informational and error messages.

Solution:

```
import logging

logging.basicConfig(filename="app.log",
                    level=logging.INFO)

logging.info("Program started")

try:
    print(10 / 0)
except ZeroDivisionError:
    logging.error("Division by zero")
```

Expected result:

Messages are saved in app.log.

15. Write a Python program that prints the content of a file and handles the case when the file is empty.

Solution:

```
with open("example.txt", "r") as file:
    data = file.read()

if data:
    print(data)
else:
    print("File is empty")
```

Expected result:

File content is printed, or File is empty.

16. Demonstrate how to use memory profiling to check the memory usage of a small program.

Solution:

```
# Install memory-profiler first if needed

from memory_profiler import profile

@profile
def test():
    numbers = [i for i in range(10000)]
    return numbers

test()
```

Expected result:

The memory profiler shows memory used by the function.

17. Write a Python program to create and write a list of numbers to a file, one number per line.

Solution:

```
numbers = [10, 20, 30, 40]

with open("numbers.txt", "w") as file:
    for n in numbers:
        file.write(str(n) + "\n")

print("Numbers saved")
```

Expected result:

Numbers saved

18. How would you implement a basic logging setup that logs to a file with rotation after 1MB?

Solution:

```
import logging
from logging.handlers import RotatingFileHandler

handler = RotatingFileHandler(
    "app.log",
    maxBytes=1024 * 1024,
    backupCount=2
)

logging.basicConfig(
    handlers=[handler],
    level=logging.INFO
)

logging.info("Program started")
```

Expected result:

Log file is rotated after about 1 MB.

19. Write a program that handles both IndexError and KeyError using a try-except block.

Solution:

```
data = {"names": ["Anu", "Riya"]}

try:
    print(data["names"][5])
except IndexError:
    print("Index error")
except KeyError:
    print("Key error")
```

Expected result:

Index error

20. How would you open a file and read its contents using a context manager in Python?

Solution:

```
with open("example.txt", "r") as file:
    data = file.read()

print(data)
```

Expected result:

The file is read and automatically closed.

21. Write a Python program that reads a file and prints the number of occurrences of a specific word.

Solution:

```
with open("example.txt", "r") as file:
    data = file.read()

word = "python"
count = data.lower().count(word)
```

```
print("Count:", count)
```

Expected result:

The number of occurrences is printed.

22. How can you check if a file is empty before attempting to read its contents?

Solution:

```
import os

if os.path.exists("example.txt"):
    if os.path.getsize("example.txt") == 0:
        print("File is empty")
    else:
        with open("example.txt", "r") as file:
            print(file.read())
```

Expected result:

File is checked before reading.

23. Write a Python program that writes to a log file when an error occurs during file handling.

Solution:

```
import logging

logging.basicConfig(filename="file_error.log",
                    level=logging.ERROR)

try:
    with open("missing.txt", "r") as file:
        print(file.read())
except OSError as e:
    logging.error("File error: %s", e)
    print("File handling error")
```

Expected result:

File handling error